CPE 314 — Software Design & Software Design Laboratory

Week 10 — Milestone 2 Grading Checklist

15% of the Finals Term Grade | Scored out of 100

This checklist verifies that every mandatory technical requirement is now implemented, even if rough around the edges: API consumption, concurrency, data mining, and visualization join the Object-Oriented Design, Data Structure / Recursion, Automated Testing, and Data Layer work already verified at Milestone 1. Week 11 is for polish and rehearsal, not new features.

Grading Checklist

#	Criterion	Description / What's Being Checked	Max Points	Points Awarded
1	API Call Succeeds & JSON Parsed/Used Meaningfully	Public REST API is called successfully from the running app — verified live, with network toggled briefly beforehand to confirm it's a real call, not cached/hard-coded — and the JSON response is parsed into real objects/fields and used meaningfully in the app, not just printed to console, confirmed by reading the response-handling source.	8	
2	Background Threading & UI Responsiveness	At least one long-running operation (large file/API/DB work) runs on a background thread — verified by triggering it directly — and the UI remains responsive/interactive while that operation runs, confirmed live by interacting with another part of the UI during the operation.	9	
3	Data Structure & Recursion Re-Verified	Stack, Queue, or Map requirement still correctly demonstrated (carried over from Milestone 1), and the genuinely recursive method is still correctly demonstrated — the group can point to both in source and trace the recursive method by hand for a small input.	6	
4	Data Mining — Real Analysis, Verified via Changed Input	A real aggregation or frequency-analysis step runs over the app's actual data, and changing the underlying input data changes both the mined result and the chart output — verified by changing an	9	

		underlying input value and confirming the result is not hard-coded.		
5	Chart Rendering & Labeling	At least one chart (bar/line/pie) renders and visibly reflects the result of the mining step, with a title, labeled axes, and a legend if more than one series is shown.	7	
6	3+ GUI Controls Wired	GUI has at least 3 distinct interactive controls, each wired to a working event handler — confirmed by clicking through all of them.	5	
7	JUnit Test Suite — Coverage & Edge Cases	6+ JUnit tests exist and pass, run live with 0 errors, and at least one test per tested class covers an edge/invalid/boundary case and asserts something meaningful — confirmed by opening the tests.	8	
8	Continued Balanced Contribution	Git history shows continued, roughly balanced contribution from every member — checked via the commit graph, flagging anyone with near-zero commits for follow-up at the Final Defense.	4	
9	Known Issues List & Requirement-to-Code Mapping	A short list of known bugs/limitations is prepared ahead of the demo and can be shown on request, and the group can point to exactly where each of the 9 mandatory requirements lives, mapped to a specific class/file in the code.	4	
10	Design Document Progress	This week's required section(s) of the team's Project Design Document are complete — see Week10_DesignDoc_Checklist.docx for the detailed breakdown	25	
11	Member Contribution (Peer-Graded)	Score derived from this member's peer-evaluation ratings from teammates (see the Peer Evaluation table below).	15	
	TOTAL		100	

Peer Evaluation — Member Contribution Scoring

Each member privately rates every teammate (not themselves) on the five criteria below, 1-5 (1 = did not contribute/unreliable, 5 = excellent). A member's 15-point Member Contribution score is the average rating received from teammates, scaled to 15: (average ÷ 5) × 15.

There are 6 identical peer-rating tables below, one per group position, in order (the 1st table is for Member 1, the 2nd for Member 2, and so on). Fill out only the ONE table matching your position in the group — for a

5-member group, one table goes unused. Write your name on the "Your Name (Rater)" line above your table, write each teammate's name into a blank column header, then rate every teammate 1-5 on each criterion below their name; do not rate yourself.

Your Name (Rater): _____

Criterion					
Preparedness & Effort					
Quality of Work					
Collaboration & Reliability					
Communication					
Meeting Deadlines / Follow-Through					

Your Name (Rater): _____

Criterion					
Preparedness & Effort					
Quality of Work					
Collaboration & Reliability					
Communication					
Meeting Deadlines / Follow-Through					

Your Name (Rater): _____

Criterion					
Preparedness & Effort					
Quality of Work					
Collaboration & Reliability					
Communication					
Meeting Deadlines / Follow-Through					

Your Name (Rater): _____

Criterion					
Preparedness & Effort					
Quality of Work					

Collaboration & Reliability					
Communication					
Meeting Deadlines / Follow-Through					

Your Name (Rater): _____

Criterion					
Preparedness & Effort					
Quality of Work					
Collaboration & Reliability					
Communication					
Meeting Deadlines / Follow-Through					

Your Name (Rater): _____

Criterion					
Preparedness & Effort					
Quality of Work					
Collaboration & Reliability					
Communication					
Meeting Deadlines / Follow-Through					

Fairness note: the instructor reviews all peer ratings before finalizing scores. Unusual patterns (e.g., everyone rating everyone a perfect 5, or one member rated far below the rest with no explanation) trigger a follow-up conversation with the group — the same discretion used for the Individual Q&A adjustment at the Final Defense.

Final Score Summary per Member

Most checklist criteria above (including Design Document Progress) are scored once for the whole group. Member Contribution (15 points, peer-graded) is the exception — it differs per student, based on the peer ratings each member receives. Use the table below to record each member's final score.

Member Name	Group/Shared Score (out of 85)	Member Contribution — Peer Score (out of 15)	Total (out of 100)
Member 1			
Member 2			

Member 3			
Member 4			
Member 5			
Member 6			

How to Submit (Git)

Milestone 2 is submitted as a branch + pull request into main, in your team's cpe314-<edp>-capstone-team-NN repo — not your individual coursework repo. See 00_GitHub_Workflow/Student_Guide_Group_Project.md for the full reference.

Step 1 — Branch

Whoever starts this checkpoint's work branches off an up-to-date main once:

```
git checkout main

git pull

git checkout -b project/milestone-2
```

Everyone else works on their own smaller feature branches day-to-day, merging into project/milestone-2 as work lands — you don't all need to work directly on project/milestone-2 itself.

Step 2 — Commit Under Your Own Account

Everyone commits from their own GitHub account, on their own machine — never one teammate typing while committing as someone else. This is what the per-member contribution scoring and the Final Defense's Individual Q&A are built on. Quick sanity check anyone can run:

```
git config user.name

git config user.email
```

Step 3 — Push and Open the Pull Request

```
git push -u origin project/milestone-2
```

On GitHub: Compare & pull request → base main → title exactly "Milestone 2 — Team NN". Open it before the deadline — that open-timestamp is your submission time; further pushes to the same PR afterward are fine.

Step 4 — Do Not Merge Your Own PR

Leave the PR open once it's submitted. Claude reviews it against this checklist's categories and posts a per-member contribution table; your instructor still does the live in-class demo/grading against this checklist before it gets merged into main.

PR Checklist

- ☐ project/milestone-2 is up to date with everything the group has merged so far
- ☐ PR title is exactly "Milestone 2 — Team NN"
- ☐ Every member's contribution is reflected in their own commits, under their own account

- ☐ PR opened before the deadline (further pushes after are fine)
- ☐ You did not merge it yourselves